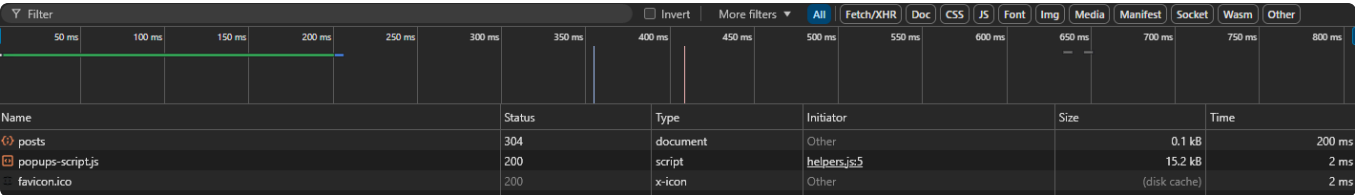


Практика 1

Выполнил Семеров Мирослав 4ИСИП-923

Загрузил страницу

Сделал скриншот списка запросов

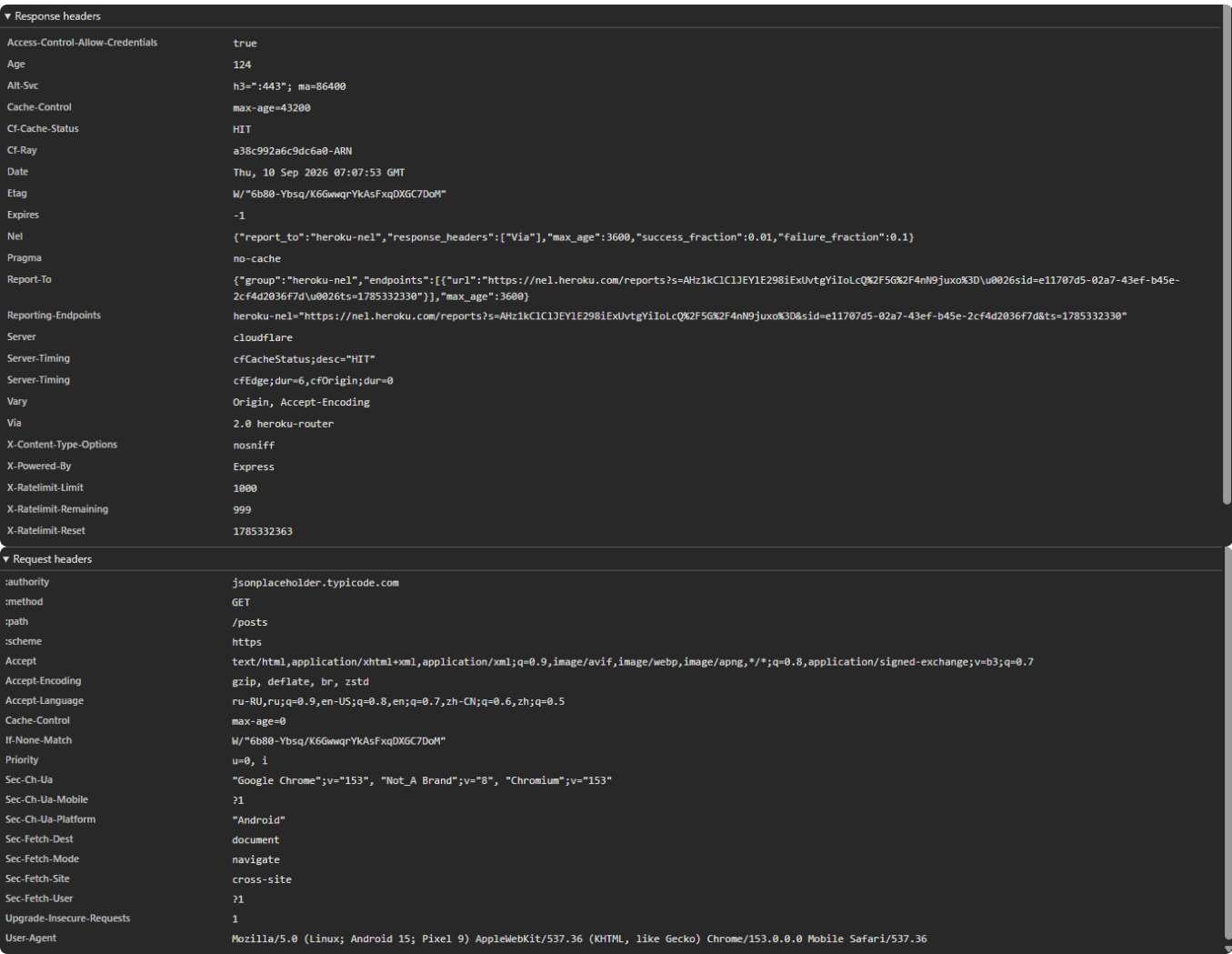


Выбираю первый запрос

метод - GET

URL и параметры - <https://jsonplaceholder.typicode.com/posts> нет параметров

заголовки -



тело ответа -

```
X Headers Preview Response Initiator Timing
▼ [-]
▶ 0: (userId: 1, id: 1, title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",-)
▶ 1: (userId: 1, id: 2, title: "qui est esse",-)
▶ 2: (userId: 1, id: 3, title: "ea molestias quasi exercitationem repellat qui ipsa sit aut",-)
▶ 3: (userId: 1, id: 4, title: "cum et est occaecati",-)
▶ 4: (userId: 1, id: 5, title: "nesciunt quas odio",-)
▶ 5: (userId: 1, id: 6, title: "dolorem eum magni eos aperiam quia",-)
▶ 6: (userId: 1, id: 7, title: "magnam facilis autem",-)
▶ 7: (userId: 1, id: 8, title: "dolorem dolore est ipsam",-)
▶ 8: (userId: 1, id: 9, title: "nesciunt iure omnis dolorem tempora et accusantium",-)
▶ 9: (userId: 1, id: 10, title: "optio molestias id quia eum",-)
▶ 10: (userId: 2, id: 11, title: "et ea vero quia laudantium autem",-)
▶ 11: (userId: 2, id: 12, title: "in quibusdam tempore odit est dolorem",-)
▶ 12: (userId: 2, id: 13, title: "dolorum ut in voluptas mollitia et saepe quo animi",-)
▶ 13: (userId: 2, id: 14, title: "voluptatem eligendi optio",-)
▶ 14: (userId: 2, id: 15, title: "eveniet quod temporibus",-)
▶ 15: (userId: 2, id: 16, title: "sint suscipit perspiciatis velit dolorum rerum ipsa laboriosam odio",-)
▶ 16: (userId: 2, id: 17, title: "fugit voluptas sed molestias voluptatem provident",-)
▶ 17: (userId: 2, id: 18, title: "voluptate et itaque vero tempora molestiae",-)
▶ 18: (userId: 2, id: 19, title: "adipisci placeat illum aut reiciendis qui",-)
▶ 19: (userId: 2, id: 20, title: "doloribus ad provident suscipit at",-)
▶ 20: (userId: 3, id: 21, title: "asperiores ea ipsam voluptatibus modi minima quia sint",-)
▶ 21: (userId: 3, id: 22, title: "dolor sint quo a velit explicabo quia nam",-)
▶ 22: (userId: 3, id: 23, title: "maxime id vitae nihil numquam",-)
▶ 23: (userId: 3, id: 24, title: "autem hic labore sunt dolores incidunt",-)
▶ 24: (userId: 3, id: 25, title: "rem alias distinctio quo quis",-)
▶ 25: (userId: 3, id: 26, title: "est et quae odit qui non",-)
▶ 26: (userId: 3, id: 27, title: "quasi id et eos tenetur aut quo autem",-)
▶ 27: (userId: 3, id: 28, title: "delectus ullam et corporis nulla voluptas sequi",-)
▶ 28: (userId: 3, id: 29, title: "iusto eius quod necessitatibus culpa ea",-)
▶ 29: (userId: 3, id: 30, title: "a quo magni similique perferendis",-)
▶ 30: (userId: 4, id: 31, title: "ullam ut quidem id aut vel consequuntur",-)
▶ 31: (userId: 4, id: 32, title: "doloremque illum aliquid sunt",-)
▶ 32: (userId: 4, id: 33, title: "qui explicabo molestiae dolorem",-)
▶ 33: (userId: 4, id: 34, title: "magnam ut rerum iure",-)
▶ 34: (userId: 4, id: 35, title: "id nihil consequatur molestias animi provident",-)
▶ 35: (userId: 4, id: 36, title: "fuga nam accusamus voluptas reiciendis itaque",-)
▶ 36: (userId: 4, id: 37, title: "provident vel ut sit ratione est",-)
▶ 37: (userId: 4, id: 38, title: "explicabo et eos deleniti nostrum ab id repellendus",-)
▶ 38: (userId: 4, id: 39, title: "eos dolorem iste accusantium est eaque quam",-)
▶ 39: (userId: 4, id: 40, title: "enim quo cumque",-)
▶ 40: (userId: 5, id: 41, title: "non est facere",-)
▶ 41: (userId: 5, id: 42, title: "commodi ullam sint et excepturi error explicabo praesentium voluptas",-)
▶ 42: (userId: 5, id: 43,-)
▶ 43: (userId: 5, id: 44, title: "optio dolor molestias sit",-)
▶ 44: (userId: 5, id: 45, title: "ut numquam possimus omnis eius suscipit laudantium iure",-)
▶ 45: (userId: 5, id: 46, title: "aut quo modi neque nostrum ducimus",-)
▶ 46: (userId: 5, id: 47, title: "quibusdam cumque rem aut deserunt",-)
▶ 47: (userId: 5, id: 48, title: "ut voluptatem illum ea doloribus itaque eos",-)
▶ 48: (userId: 5, id: 49, title: "laborum non sunt aut ut assumenda perspiciatis voluptas",-)
▶ 49: (userId: 5, id: 50,-)
▶ 50: (userId: 6, id: 51, title: "soluta aliquam aperiam consequatur illo quis voluptas",-)
▶ 51: (userId: 6, id: 52, title: "qui enim et consequuntur quia animi quis voluptate quibusdam",-)
▶ 52: (userId: 6, id: 53, title: "ut quo aut ducimus alias",-)
▶ 53: (userId: 6, id: 54, title: "sit asperiores ipsam eveniet odio non quia",-)
▶ 54: (userId: 6, id: 55, title: "sit vel voluptatem et non libero",-)
▶ 55: (userId: 6, id: 56, title: "qui et at rerum necessitatibus",-)

1 [
2 {
3   "userId": 1,
4   "id": 1,
5   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7 },
8 {
9   "userId": 1,
10  "id": 2,
11  "title": "qui est esse",
12  "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis poss
13 },
14 {
15   "userId": 1,
16   "id": 3,
17   "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18   "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
19 },
20 {
21   "userId": 1,
22   "id": 4,
23   "title": "cum et est occaecati",
24   "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum i
25 },
26 {
27   "userId": 1,
28   "id": 5,
29   "title": "nesciunt quas odio",
30   "body": "repudiandae veniam quaerat sunt sed\nnalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque"
31 },
32 {
33   "userId": 1,
34   "id": 6,
35   "title": "dolorem eum magni eos aperiam quia",
36   "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et dolorem
37 },
38 {
39   "userId": 1,
40   "id": 7,
41   "title": "magnam facilis autem",
42   "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
43 },
44 {
45   "userId": 1,
46   "id": 8,
47   "title": "dolorem dolore est ipsam",
48   "body": "dignissimos aperiam dolorem qui eum\nfacilis quibusdam animi sint suscipit qui sint possimus cum\nquaerat magni maiores excepturi\nipsam ut commodi dolor voluptatum modi aut vitae"
49 },
50 {
51   "userId": 1,
52   "id": 9,
```

код ответа - 304

время выполнения - 196 ms

Эксперименты с запросами

метод - get

url и параметры - <https://jsonplaceholder.typicode.com/posts/1> параметра нет

заголовки -

Response headers	
Access-Control-Allow-Credentials	true
Age	18609
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	a38c9d636f50c6a0-ARN
Date	Thu, 10 Sep 2026 07:10:46 GMT
Etag	W/"124-yiKdLzqO5gFBrJFrcdJ8Yq8LGnU"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=vm67FVLNhsCgrFgubRa04ooDeMKdgwXS9H3i2IbjuoYK3D\u0026ts=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1785194657"}], "max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=vm67FVLNhsCgrFgubRa04ooDeMKdgwXS9H3i2IbjuoYK3D\u0026ts=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1785194657"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=5,cfOrigin;dur=0
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	999
X-Ratelimit-Reset	1785194663
Request headers	
authority	jsonplaceholder.typicode.com
method	GET
path	/posts/1
scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7,zh-CN;q=0.6,zh;q=0.5
Cache-Control	max-age=0
If-None-Match	W/"124-yiKdLzqO5gFBrJFrcdJ8Yq8LGnU"
Priority	u=0, i
Sec-Ch-UA	"Google Chrome";v="153", "Not A Brand";v="8", "Chromium";v="153"
Sec-Ch-UA-Mobile	?1
Sec-Ch-UA-Platform	"Android"

тело ответа -

Preview	
<pre>{ "userId": 1, "id": 1, "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit", "body": "quia et suscipit\nsuscepit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto" }</pre>	

код ответа - 304

время выполнения - 170 ms

Выводы между двумя запросами

Размер ответа - все посты(7,5kb) > 1 пост(1kb)

Время загрузки - все посты(196ms) > 1 пост(170ms)

Количество записей - все посты > 1 пост

Вывод. Все посты занимают больше времени больше весят и выдают больше записей

Работа с параметрами

В <https://jsonplaceholder.typicode.com/posts?userId=1> вернулось 10 объектов размер 1.2kb время 195ms (200)

В <https://jsonplaceholder.typicode.com/posts> вернулось 100 объектов размер 7.2 kb время 183 ms (304)

Вывод : параметры улучшают выборку данных и выдают более конкретные результаты за счет чего уменьшают размер

Массовые запросы

Время запросов примерно одинаковое

Избыточных запросов нет потому что у нас простые методы, которые спокойно обрабатываются и не наслаиваются друг на друга

Все запросы отдаются по HTTP 2 и используют одно и то же соединение поэтому все идет отдельными потоками одновременно.

Заголовки и ответы

Выписываю 5 ключевых заголовков

Server cloudfare - сервер

Date Wed, 09 Sep 2026 11:45:15 GMT - дата

via 2.0 heroku-router - отслеживает ip в данном случае устройство

pragma no-cache - заголовок для управления кешированием

age 16835 - возраст страницы

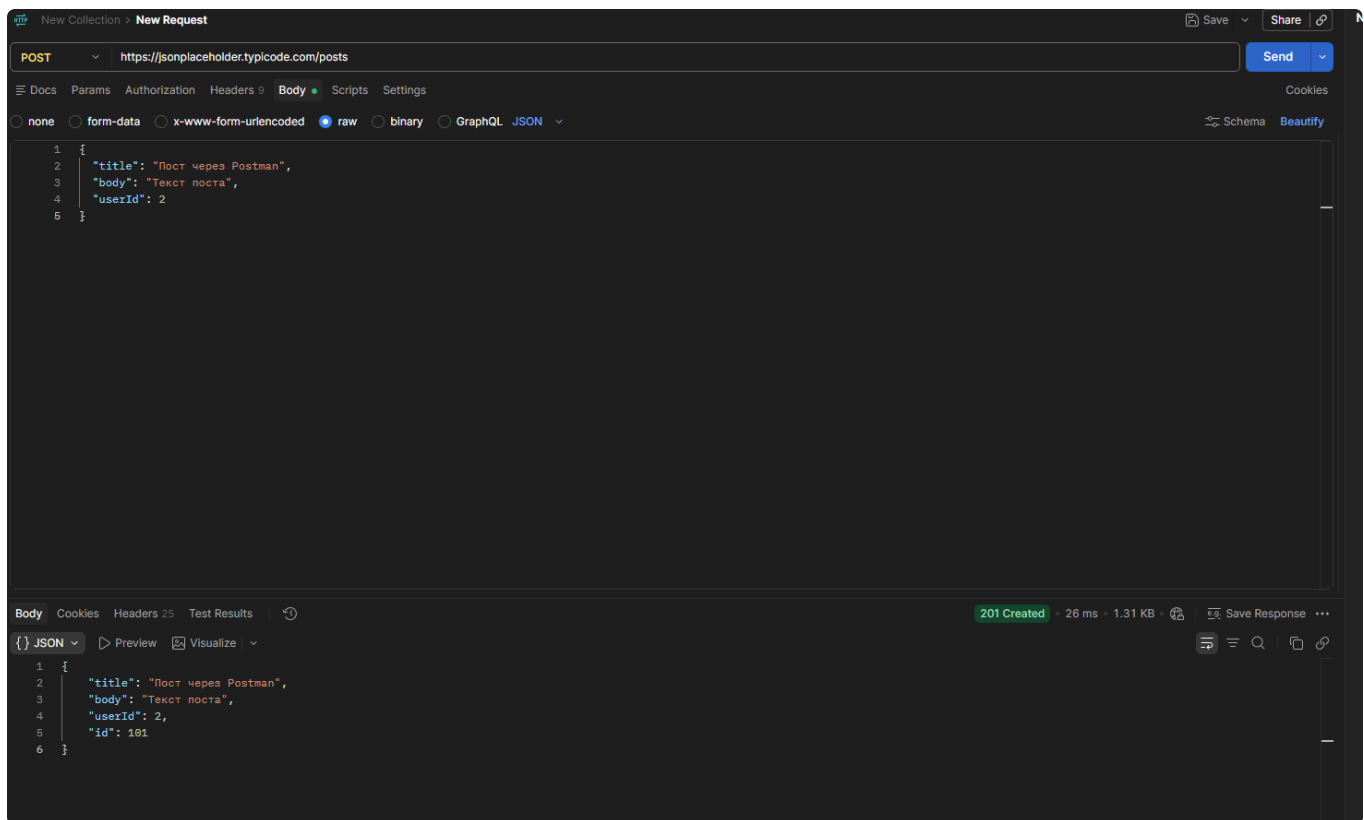
Сравните вкладки **Preview** и **Response**:

Превью это то как видит пользователь и респонс это то как видит его компьютер. В предоставлении данных отличаются ведь компьютер и человек понимают по разному например картинки передаются в респонс через двоичный код. Превью удобнее использовать когда мы хотим показать пользователю а респонс это когда мы хотим донести до компьютера

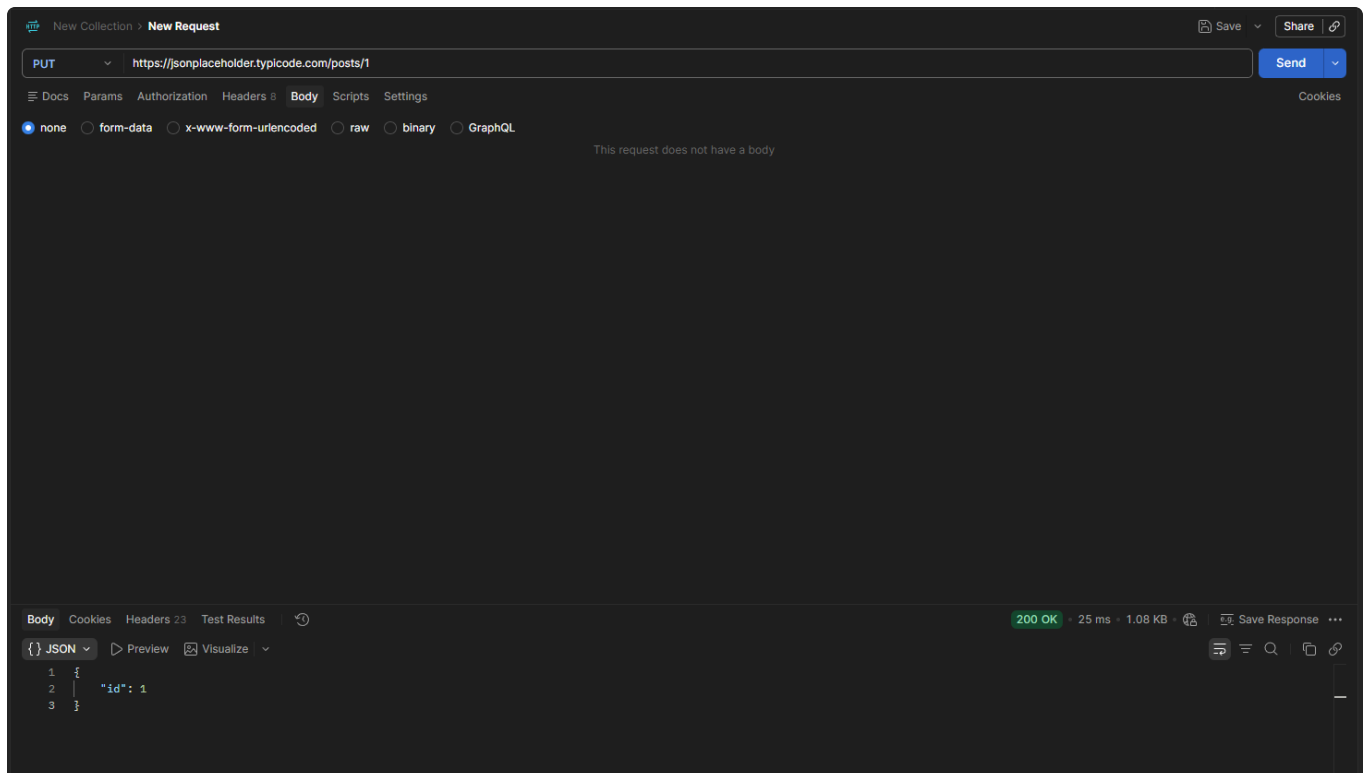
Работа с API через Postman

Выполнил get posts он выдал 100 значений, а через posts/1 выдал всего одну запись.

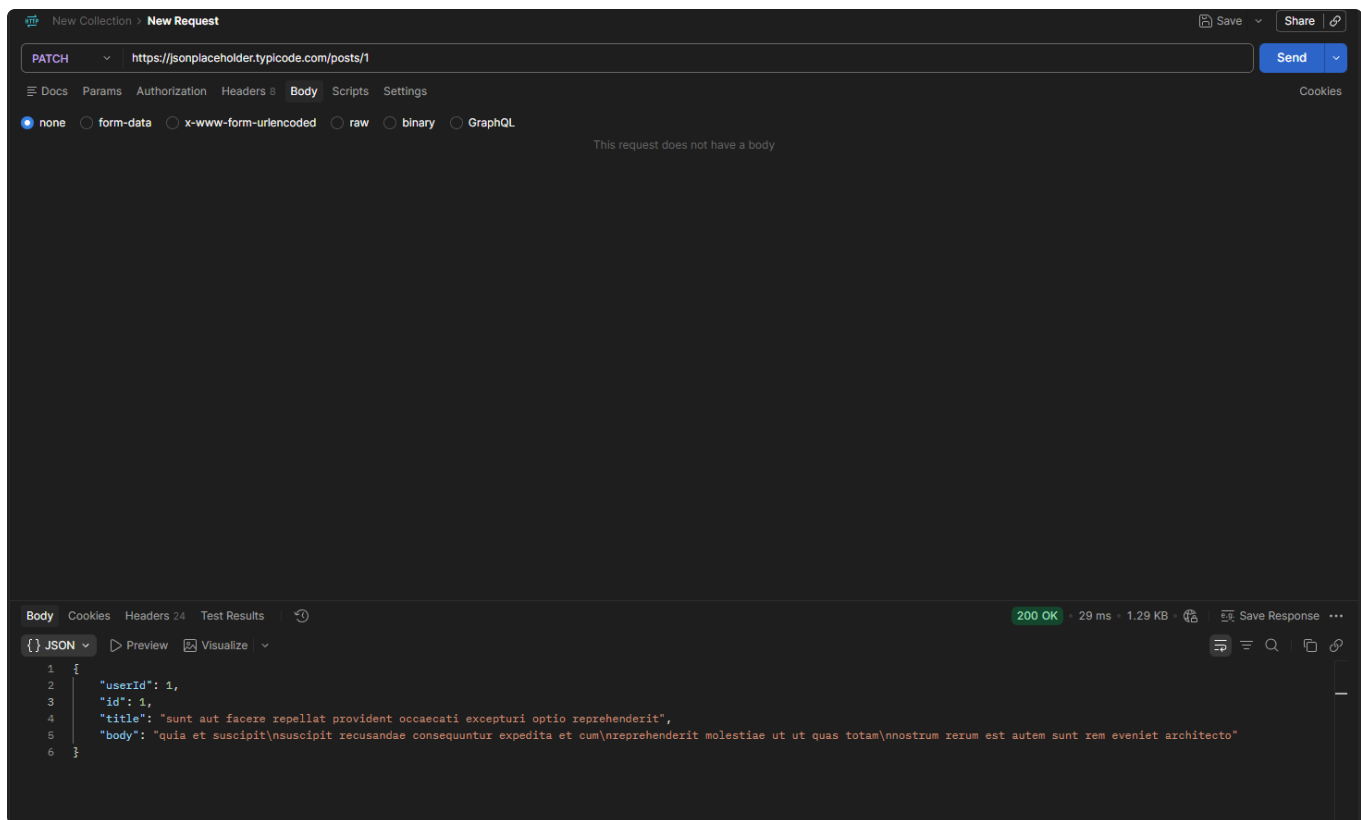
Выполнил POST с JSON телом



Выполнил PUT через posts/1



Выполнил PATCH через posts/1



Происходят разные действия когда мы пишем put мы предполагаем полную замену той части и ресурса в которую мы вторгаемся так как мы делаем это с пустой консолью и ничего в put не закладываем он просто возвращает пустой результат так как отчистил запрос и ничем не заменил. а PATCH ожидает конкретных параметров которые мы хотим заменить и когда мы делаем уже его с пустым запросом сервер просто видит что заменять нечего и возвращает исходный результат.

Выполнил DELETE через posts/1

Код ответа - 200

Тело ответа - пустое. метод Delete просто стер весь пост под этим id

Сравнение DevTools и Postman

Devtools удобнее когда нам нужно понять что на самом деле происходит на сайте и узнать все о реальной производительности наших запросов на реальном примере и на нем легче отлаживать проблемы когда есть конкретная задача. DevTools хардкорный инструмент под конкретные задачи. Postman чуть более творческий тут мы можем сами что-то разрабатывать и тестировать, также очень удобно повторять и сохранять запросы. Postman больше инструмент для создания и тестирования, а DevTools помогает посмотреть как конкретно ведут себя запросы на боевых примерах.

Контрольные вопросы

1. Что такое DevTools и какие задачи решают вкладки Elements, Console и Network?
DevTools — встроенные инструменты разработчика в браузере (F12). Elements —

смотришь и правишь HTML/CSS страницы прямо в браузере. Console — там ошибки скриптов и можно писать JS-код вручную. Network — видно все запросы, которые ушли со страницы (какие, куда, сколько по времени).

2. Как открыть DevTools и какие настройки вкладки Network важны для замеров производительности?

F12 или ПКМ → "Просмотреть код" (Inspect). Для замеров важно: включить "Disable cache" (чтобы не мешал старый кэш), "Preserve log" (чтобы лог не стирался при переходах), смотреть колонку Time и водопад (waterfall) по каждому запросу.

3. Перечислите основные части HTTP-запроса (метод, URL/параметры, заголовки, тело) и их назначение.

- Метод (GET, POST...) — что делаем с ресурсом.
- URL/параметры — куда идём и с какими доп. данными (?id=1).
- Заголовки — мета-инфа (тип контента, авторизация и т.п.).
- Тело — сами данные, которые отправляем (обычно в POST/PUT/PATCH).

4. Чем отличаются методы GET, POST, PUT, PATCH и DELETE? Приведите по одному практическому примеру.

- GET — получить данные (открыть страницу товара).
- POST — создать новое (зарегистрировать юзера).
- PUT — полностью заменить ресурс (обновить весь профиль).
- PATCH — частично обновить (поменять только email).
- DELETE — удалить (удалить пост).

5. Для чего используются методы HEAD и OPTIONS и где встречается preflight-запрос? HEAD — как GET, но без тела ответа, просто узнать заголовки/статус (например, существует ли файл). OPTIONS — узнать, какие методы разрешены сервером. Preflight — это как раз автоматический OPTIONS-запрос, который браузер шлёт перед "сложным" CORS-запросом (с кастомными заголовками или методами типа PUT/DELETE), чтобы спросить разрешения у сервера.

6. Расскажите о группах кодов ответов (1xx–5xx) и приведите примеры для 200, 301/302, 304, 404, 500.

- 1xx — информационные (редко видим).
- 2xx — успех (200 — OK, всё норм).
- 3xx — редиректы (301 — переехал навсегда, 302 — временно, 304 — не изменилось, бери из кэша).
- 4xx — ошибка клиента (404 — не найдено).
- 5xx — ошибка сервера (500 — сервер упал/сломался).

7. Что такое REST и как ресурс в REST представлен на уровне URL?

REST — архитектурный стиль API, где всё представлено как "ресурсы", а действия

над ними — через HTTP-методы. Ресурс в URL — это обычно существительное во множественном числе + id: `/posts/1`, `/users/5`.

8. Какие типы данных поддерживает JSON? Приведите короткий пример JSON-объекта и массива.

JSON поддерживает: строку, число, `boolean`, `null`, объект, массив.

```
{ "name": "Ivan", "age": 20, "isStudent": true, "hobbies": ["код", "сон"] }
```

9. В чём отличие вкладок Preview и Response в карточке запроса DevTools?

Response — сырой ответ сервера как есть (текст/JSON без форматирования). Preview — та же информация, но красиво отформатированная и раскрываемая для пользователя

10. Какие заголовки чаще всего анализируются при оптимизации (например, Content-Type, Cache-Control, ETag, Authorization)?

Content-Type (что за тип данных пришёл), Cache-Control (как долго можно кэшировать), ETag (не изменился ли файл, для 304), Authorization (токен доступа).

11. Какие приёмы снижения нагрузки на сеть можно использовать для API: пагинация, фильтрация, кэширование и др.?

Пагинация (отдавать данные порциями, не всё сразу), фильтрация/сортировка на сервере (не тащить лишнее), кэширование (не запрашивать одно и то же повторно), сжатие ответа (gzip), использование HTTP/2 (мультиплексирование запросов).

12. Чем работа с API в Postman отличается от анализа запросов в DevTools, и когда уместен каждый инструмент?

DevTools смотрит на запросы, которые уже реально ушли со страницы сайта — хорош для анализа и отладки живого сайта. Postman — сам конструируешь запросы вручную, сохраняешь их в коллекции, можно писать тесты — хорош для разработки и тестирования API отдельно от фронтенда.